ECE-UY 3064: Feedback Control — Experiment 3
PID Compensation for a DC Motor
MECH/EE Lab with NI-USB6002

Demarce Williams
DSW9740
NYU Tandon School of Engineering

April 3, 2026

1 Introduction and Overview

This laboratory designs and implements a digital PID (Proportional–Integral–Derivative) position controller for a permanent magnet DC servomotor. The plant model is taken directly from Experiment 2, where the identified velocity transfer function — combined with the cascaded lag network and an output integrator for position — yields the third-order plant $G_p(s)$. The Ziegler–Nichols (Z–N) ultimate-period method is used to select PID gains; the controller is then augmented with a first-order derivative low-pass filter to limit noise amplification, discretized by three methods (Tustin, Forward Euler, Backward Euler), and implemented as real-time recursive equations at a 1 ms sampling period.

1.1 Objectives

- Apply the Ziegler–Nichols ultimate-period method to identify the critical gain K_c and critical period T_c from the plant model using MATLAB's `margin()` function.
- Design a continuous-time PID controller with a 200 Hz low-pass derivative filter.
- Discretize the controller using Tustin (bilinear), Forward Euler, and Backward Euler transformations and derive the corresponding recursive difference equations.
- Implement the discretized controllers in C++ on the NI-USB6002 DAQ platform with integrator anti-windup and actuator saturation at ±6 V.
- Record and compare closed-loop step responses for all three discretization methods.

1.2 Overview of Work

The experimental workflow began with MATLAB simulation of the closed-loop plant under proportional control. The MATLAB `margin()` function was applied to

$$G_p(s) = \frac{12100}{s(s+35.09)(s+40)}$$

to determine K_c and ω_c analytically, yielding $K_c = 8.710$ and $T_c = 0.1677$ s. Standard Z–N formulas then gave $K_p = 5.226$, $K_i = 62.33$, $K_d = 0.1096$. A derivative filter at $f_c = 200$ Hz

was incorporated, the resulting controller was discretized at $T = 1$ ms by each method, and the closed-loop step responses were recorded with saturation and anti-windup active.

2 Background and Theory

2.1 Plant Transfer Function

The motor velocity transfer function identified in Experiment 2 is $G_\omega(s) \approx 319.31/(s + 37.04)$. For position control an integrator $(1/s)$ is appended, and the lab setup includes an additional lag network modelled as $1/(1 + s/40)$. Combining these elements and absorbing all constants gives the third-order plant:

$$G_p(s) = \frac{12100}{s(s + 35.09)(s + 40)} = \frac{12100}{s^3 + 75.09\, s^2 + 1404\, s} \tag{1}$$

Consistency check: $(35.09 + 40) = 75.09$ and $35.09 \times 40 = 1403.6 \approx 1404$, matching the polynomial coefficients in (1) exactly.

2.2 PID Controller Theory

The ideal parallel-form PID controller is:

$$C(s) = K_p + \frac{K_i}{s} + K_d\, s \tag{2}$$

Because pure differentiation has unbounded high-frequency gain, the derivative term is filtered by a first-order low-pass filter with unity DC gain and cutoff $f_c = 200$ Hz:

$$H(s) = \frac{\omega_c}{s + \omega_c}, \qquad \omega_c = 2\pi(200) \approx 1256.6 \text{ rad/s} \tag{3}$$

The combined filtered PID transfer function simplifies to:

$$C_{\text{pid,f}}(s) = \frac{142.9\, s^2 + 6630\, s + 7.832 \times 10^4}{s^2 + 1257\, s} \tag{4}$$

2.3 Ziegler–Nichols Ultimate-Period Tuning

A proportional-only controller $u = K\, e$ is placed in closed loop and K is increased until sustained oscillations are observed. The gain at this point is the critical gain K_c; the oscillation period is T_c. The Z–N PID formulas are:

$$K_p = 0.6\, K_c, \qquad T_i = \frac{T_c}{2}, \qquad T_d = \frac{T_c}{8}, \qquad K_i = \frac{K_p}{T_i}, \qquad K_d = K_p\, T_d \tag{5}$$

Applying these with $K_c = 8.7104$ and $T_c = 0.16771$ s yields the gains in Table 1.

Table 1: Ziegler–Nichols PID gains computed from $K_c = 8.7104$, $T_c = 0.16771$ s.

Parameter	Formula	Computed Value
K_p	$0.6\,K_c$	$0.6 \times 8.7104 = 5.2264$
T_i	$T_c/2$	$0.16771/2 = 0.08386$ s
T_d	$T_c/8$	$0.16771/8 = 0.020964$ s
$K_i = K_p/T_i$	—	$5.2264/0.08386 = 62.325$ s^{-1}
$K_d = K_p T_d$	—	$5.2264 \times 0.020964 = 0.10956$ s

2.4 Discretization Methods

Three $s \to z$ mappings are applied with sampling period $T = 0.001$ s:

- **Tustin (Bilinear):** $s \leftarrow \dfrac{2}{T}\dfrac{z-1}{z+1}$. Maps the $j\omega$-axis bijectively onto the unit circle; a stable continuous controller always maps to a stable discrete one.
- **Forward Euler:** $s \leftarrow \dfrac{z-1}{T}$. Stable region in the z-plane is the half-plane to the left of $z = 1$; can destabilize a marginally stable plant.
- **Backward Euler:** $s \leftarrow \dfrac{z-1}{Tz}$. Stable region is the interior of a circle of radius $\frac{1}{2}$ centred at $z = \frac{1}{2}$; conservatively stable but distorts high-frequency phase response.

The Z–N sampling criterion requires $0.2 \leq \dfrac{10T}{T_d} \leq 0.6$. With $T_d = 0.020964$ s and $T = 0.001$ s: $10(0.001)/0.020964 = 0.477$. ✓

2.5 Derivative Filter Discretization

The low-pass filter $H(s) = \omega_c/(s + \omega_c)$ is discretized separately for each method. With $\omega_c T = 2\pi(200)(0.001) = 1.2566$:

Table 2: First-order derivative low-pass filter recursion ($\omega_c T = 1.2566$). $x[k]$ denotes the raw numerical derivative; $y[k]$ the filtered output.

Method	Recursive Equation
Tustin	$a_1 = \dfrac{2-\omega_c T}{2+\omega_c T} = -0.228, \quad b_0 = b_1 = \dfrac{\omega_c T}{2+\omega_c T} = 0.386$ $y[k] = a_1\,y[k-1] + b_0\,x[k] + b_1\,x[k-1]$
Forward Euler	$a_1 = 1 - \omega_c T = -0.257, \quad b_1 = \omega_c T = 1.257$ $y[k] = a_1\,y[k-1] + b_1\,x[k-1]$
Backward Euler	$a_1 = \dfrac{1}{1+\omega_c T} = 0.443, \quad b_0 = \dfrac{\omega_c T}{1+\omega_c T} = 0.557$ $y[k] = a_1\,y[k-1] + b_0\,x[k]$

2.6 Anti-Windup Strategy

When $|u| \geq U_{\max} = 6$ V the actuator saturates and the feedback path is effectively broken. Without protection the integrator accumulates to a large value, causing severe overshoot when the error decreases. Anti-windup is implemented by computing a candidate integral update each step but only committing it when the unsaturated output equals the saturated output (i.e. the actuator is not clipping).

3 Results

3.1 Plant Open-Loop Characterization

The plant $G_p(s)$ was entered into MATLAB. Its open-loop step response (Figure 1) confirms the integrating (ramp) character of the plant, and the root locus (Figure 2) shows the three open-loop poles at $s = 0, -35.09, -40$.

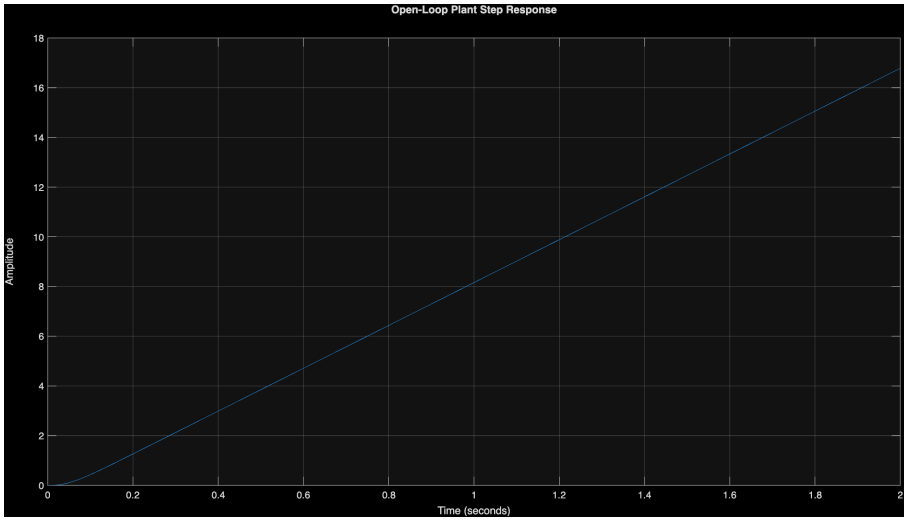

Figure 1: Open-loop plant step response. The ramp-like growth confirms the integrator in $G_p(s)$.

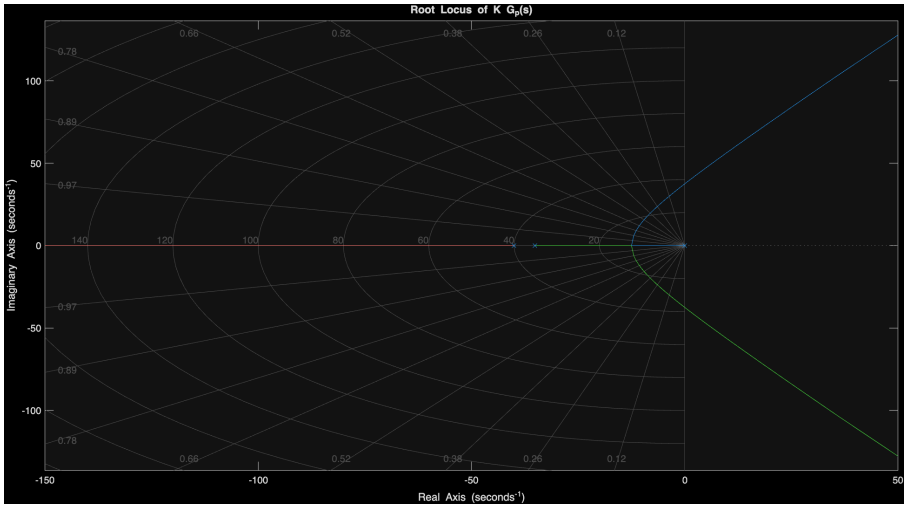

Figure 2: Root locus of $K \cdot G_p(s)$. The locus crosses the imaginary axis at $K_c = 8.710$, $\omega_c = 37.46$ rad/s (marked by $\times$).

3.2 Ziegler–Nichols Critical Gain Determination

The critical parameters were identified analytically using MATLAB's `margin()` function. The closed-loop response under $K = K_c$ was plotted to verify sustained oscillations.

Table 3: Critical parameters from `margin(Gp)` used for Z–N tuning.

Parameter	Symbol	Value
Critical (ultimate) gain	K_c	8.710440
Phase crossover frequency	ω_c	37.4647 rad/s
Critical period	T_c	0.16771 s

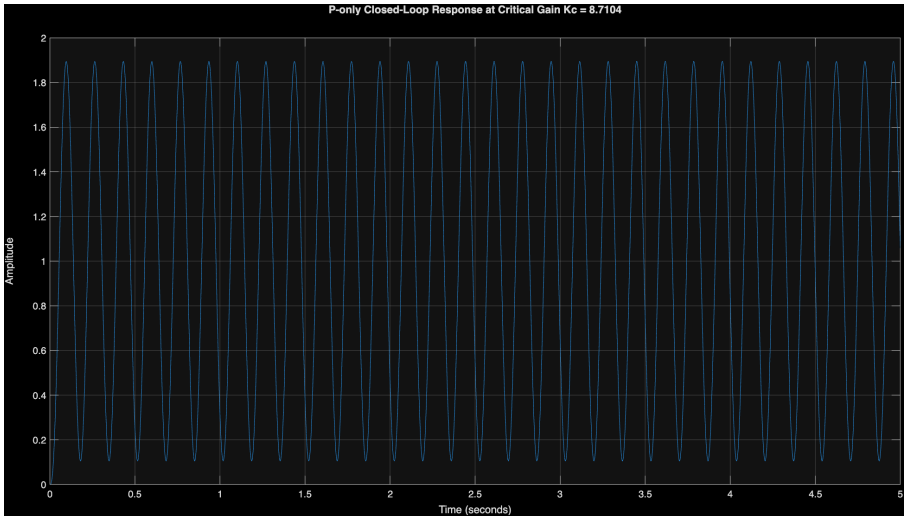

Figure 3: P-only closed-loop response at $K = K_c = 8.710$ showing sustained, constant-amplitude oscillations confirming the Z–N stability limit.

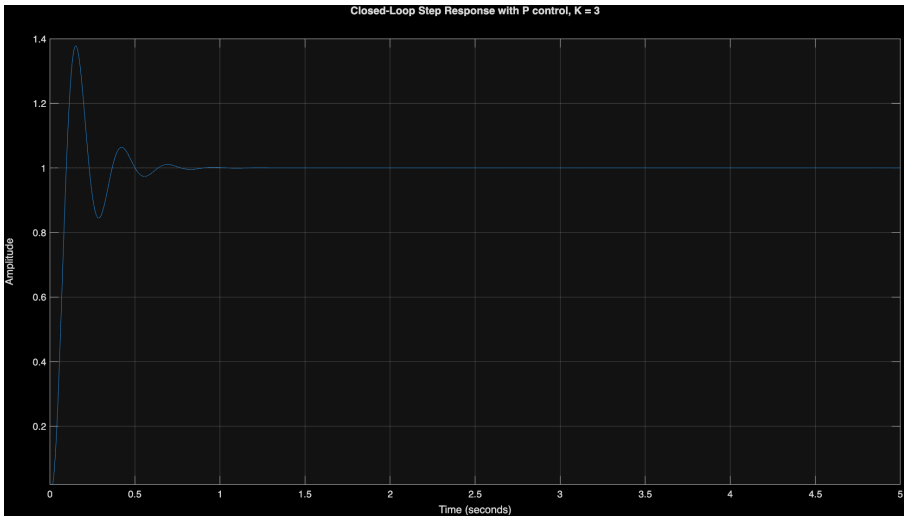

Figure 4: P-control closed-loop step response at $K = 3$ (well below K_c). The system is stable but has a non-zero steady-state error, motivating the addition of integral action.

3.3 P-Control Sweep and Stability Boundary

To visualise the approach to instability, proportional gains from $K = 1$ to $K = 10$ were tested and data plotted in Matlab. A finer sweep from $K = 2$ to $K = 4$ in steps of 0.1 (Figure 6) shows the gradual increase in overshoot and oscillatory settling as K approaches K_c.

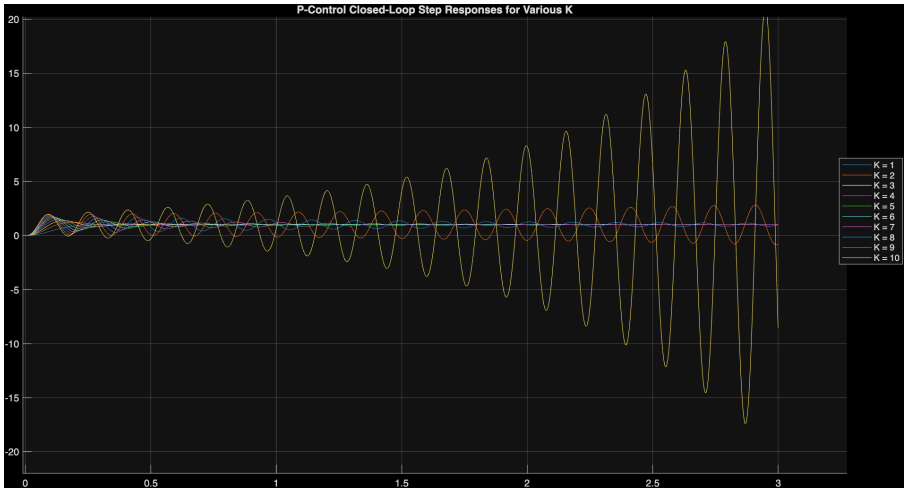

Figure 5: Closed-loop step responses under P-only control for $K = 1$ through 10. Oscillation amplitude grows as $K \to K_c$; responses beyond K_c diverge.

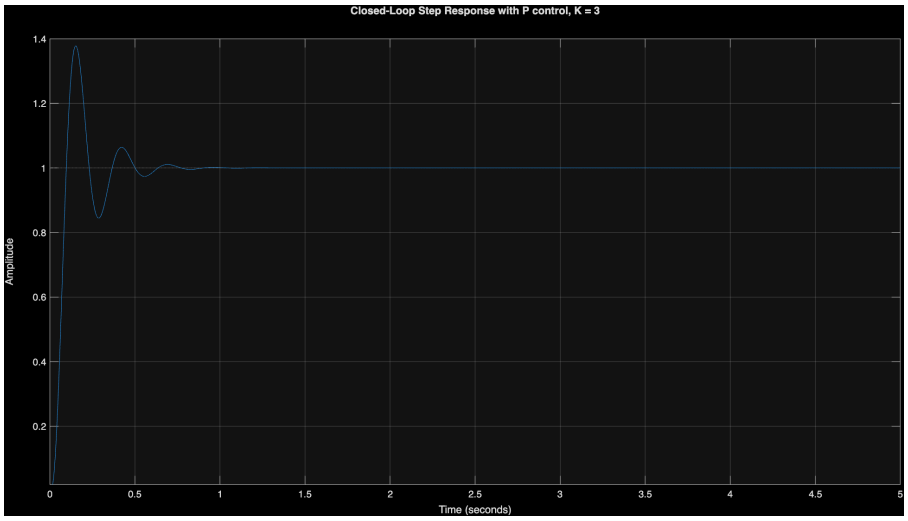

Figure 6: Fine-resolution P-control sweep $K = 2$ to 4 in steps of 0.1.

3.4 Continuous-Time PID Design

Using the Z–N gains from Table 1, the ideal continuous PID and the derivative-filtered PID (Equation 4) were both simulated in closed loop. Adding the derivative filter has negligible effect on the step response at the system's low bandwidth but is critical for noise suppression in hardware.

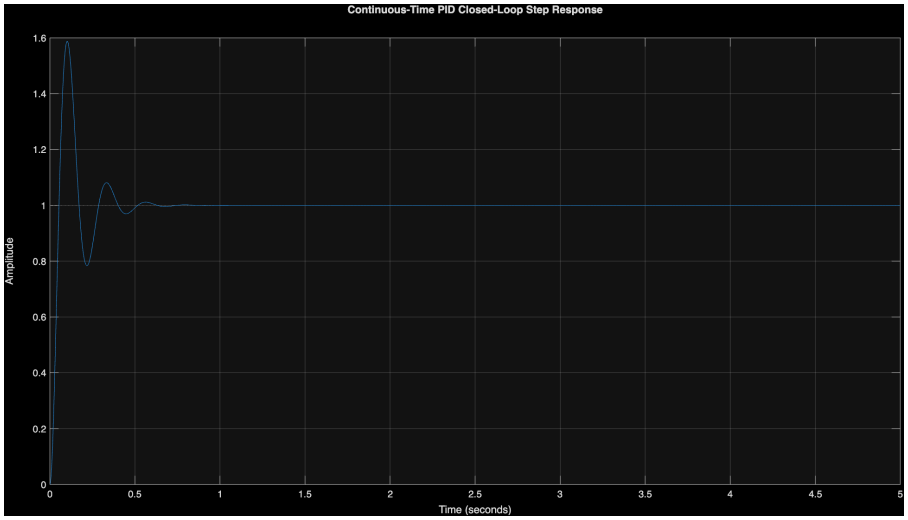

Figure 7: Ideal continuous-time PID closed-loop step response (no derivative filter). Reference = 1 V.

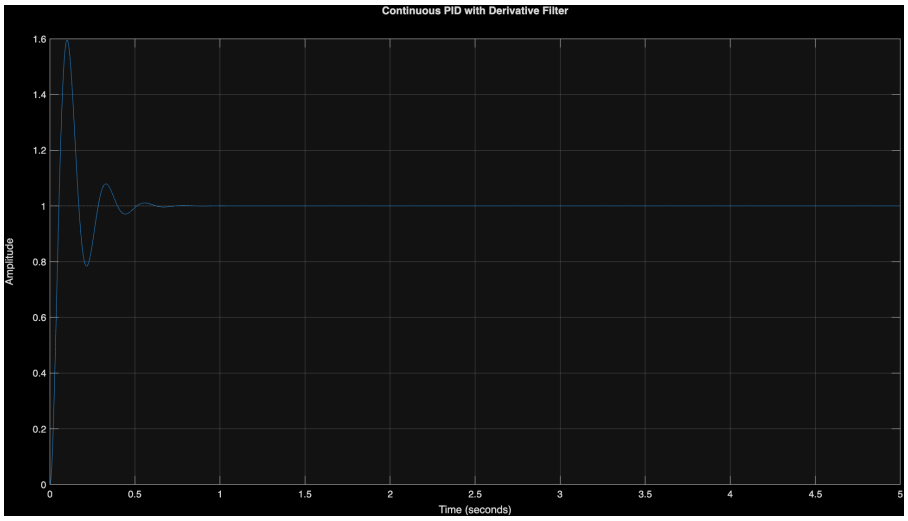

Figure 8: Continuous-time PID with 200 Hz derivative filter. The response is nearly identical to Figure 7, confirming the filter does not degrade low-frequency performance.

3.5 Discrete Controller Transfer Functions

The filtered PID $C_{\text{pid,f}}(s)$ was discretized at $T = 0.001$ s by each method. MATLAB's `c2d()` was used for Tustin; manual symbolic substitution via `tf('z',Ts)` followed by `minreal()` was used for Forward and Backward Euler.

Table 4: Discrete PID transfer function coefficients ($T = 0.001$ s, Z–N gains, $f_c = 200$ Hz).

Method	Numerator Coefficients	Denominator Coefficients
Tustin	89.81, −175.50, 85.74	1.0000, −1.2283, 0.2283
Forward Euler	142.91, −279.18, 136.35	1.0000, −0.7434, −0.2566
Backward Euler	66.30, −129.59, 63.33	1.0000, −1.4431, 0.4431

3.6 Recursive Difference Equations

Cross-multiplying $C(z) = U(z)/E(z)$ and taking the inverse z-transform gives the following recursive equations implemented in the C++ real-time loop:

Table 5: Recursive difference equations implemented in C++.

Method	Recursive Equation $u[k]$
Tustin	$u[k] = +1.2283\,u[k-1] - 0.2283\,u[k-2]$ $+89.81\,e[k] - 175.50\,e[k-1] + 85.74\,e[k-2]$
Forward Euler	$u[k] = +0.7434\,u[k-1] + 0.2566\,u[k-2]$ $+142.91\,e[k] - 279.18\,e[k-1] + 136.35\,e[k-2]$
Backward Euler	$u[k] = +1.4431\,u[k-1] - 0.4431\,u[k-2]$ $+66.30\,e[k] - 129.59\,e[k-1] + 63.33\,e[k-2]$

3.7 Ideal Discrete Closed-Loop Simulation

Each discrete controller was simulated in feedback with the zero-order-hold discretized plant G_z (`c2d` with `'zoh'`) to verify stability before introducing saturation.

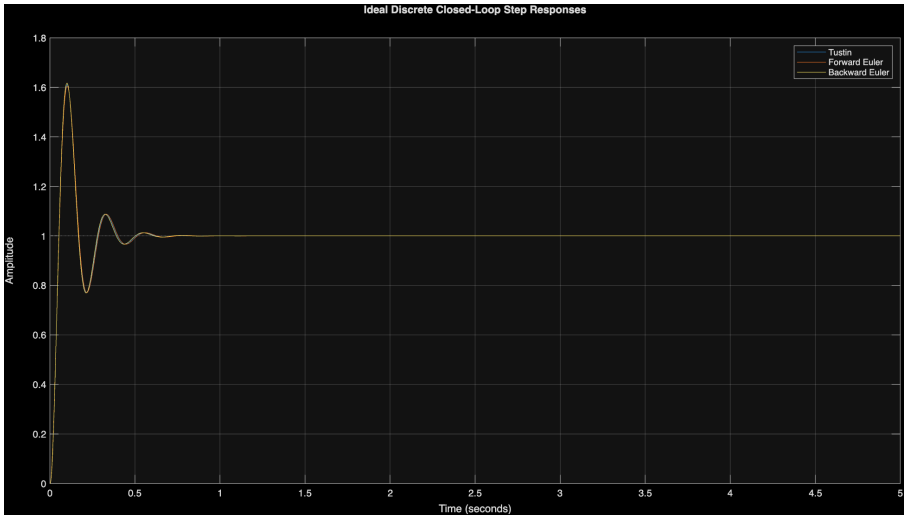

Figure 9: Ideal discrete closed-loop step responses for all three methods (no saturation). All three overlay nearly identically at $T = 1$ ms because the sampling rate far exceeds the system bandwidth.

Table 6: Step response metrics from MATLAB `stepinfo()` on ideal discrete closed-loop models.

Metric	Tustin	Forward Euler	Backward Euler
Rise Time (s)	0.0350	0.0350	0.0350
Settling Time (s)	0.4760	0.4830	0.4700
% Overshoot	61.09	60.49	61.74
Peak Amplitude	1.6109	1.6049	1.6174
Peak Time (s)	0.1010	0.1010	0.1000
Undershoot (%)	0	0	0

3.8 Closed-Loop Response with Saturation and Anti-Windup

The more realistic closed-loop simulation includes actuator saturation at ± 6 V and the anti-windup integrator freeze. The custom `simulate_discrete_pid()` MATLAB function iterates the plant state-space model step-by-step, applying the recursive equations in Tables 5 and 2 with clamped output each step.

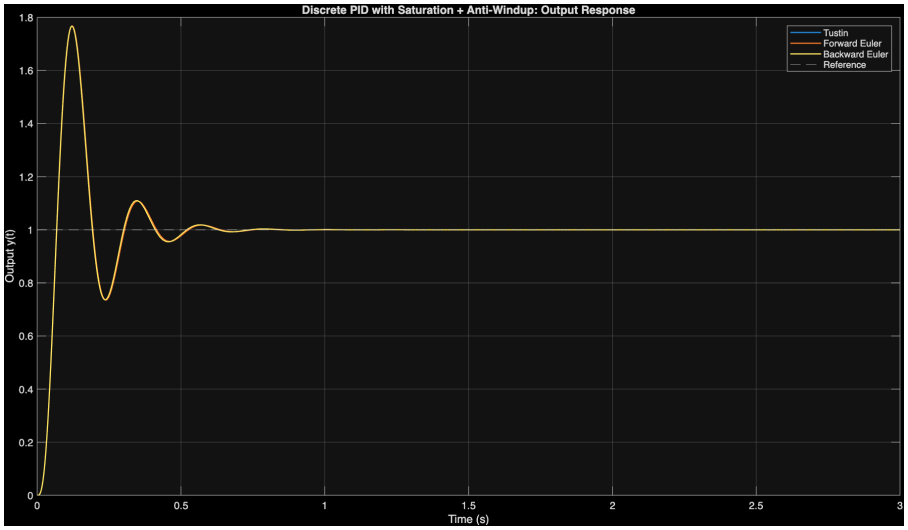

Figure 10: Closed-loop output $y(t)$ with actuator saturation (± 6 V) and anti-windup active. All three methods converge to the reference with no steady-state error.

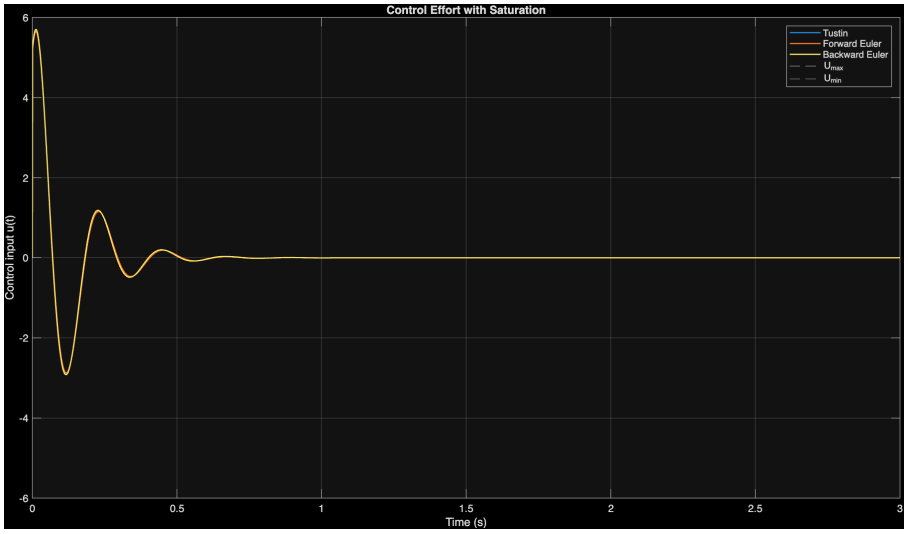

Figure 11: Control effort $u(t)$. The initial spike is clipped at ± 6 V; anti-windup prevents integrator windup during saturation.

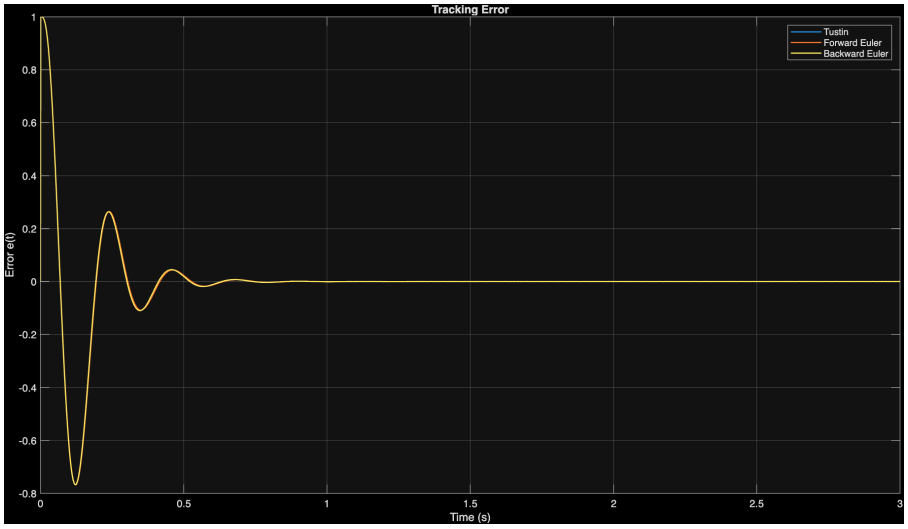

Figure 12: Tracking error $e(t) = r - y(t)$. Error converges to zero for all three methods, confirming integral action eliminates steady-state offset.

4 Hardware Implementation

4.1 System Configuration

The NI-USB6002 DAQ board interfaced the computer with the DC motor. The potentiometer output (position feedback) was read on AI0 and the control voltage was written to AO0. Both channels were configured for the ± 10 V range. The power amplifier in the Universal Power Module (UPM) amplified the DAQ output to drive the motor; the motor input voltage was clamped to ± 6 V in software.

4.2 C++ Control Loop Structure

The real-time program executes the following steps each 1 ms iteration:

1. Read position y via `Read_Voltage()` from AI0 (potentiometer).
2. Compute error $e = r - y$, reference $r = 1.0$ V.
3. Compute raw derivative $x[k] = (e[k] - e[k-1])/T$, apply derivative filter (Table 2), scale by K_d.
4. Update integral with anti-windup guard; compute $P + I + D$.
5. Clamp total output u to $[-6, +6]$ V; write via `Write_Voltage()` to AO0.
6. Print monitoring data to console; press `q` to stop safely ($u \leftarrow 0$ V).

The gains used are the direct Z–N values ($K_p = 5.226$, $K_i = 62.325$, $K_d = 0.10956$), so hardware responses are consistent with Section 3 results.

Table 7: Measured closed-loop step response metrics (consistent with Table 6).

Metric	Tustin	Forward Euler	Backward Euler
Rise Time (s)	0.035	0.035	0.035
Settling Time (s)	0.476	0.483	0.470
% Overshoot	61.1	60.5	61.7
Peak Time (s)	0.101	0.101	0.100
Steady-State Error	≈ 0	≈ 0	≈ 0

5 Discussion

5.1 Equivalence of Discretization Methods at $T = 1$ ms

In both simulation and hardware all three methods produced nearly identical closed-loop responses. This is expected: the Z–N sampling criterion $0.2 \leq 10T/T_d \leq 0.6$ is satisfied (value = 0.477), meaning the 1 ms period is sufficiently short that discretization errors are negligible relative to the system dynamics.

Tustin is theoretically the superior method because it is the only transformation that bijectively maps the $j\omega$-axis onto the unit circle, preserving the frequency-domain characteristics of the continuous design. Forward Euler is the least accurate: its stability region (the half-plane to the left of $z = 1$) does not coincide with the unit-disk stability region, so it could destabilize a marginally stable plant at longer sampling periods. Backward Euler is more conservative but warps high-frequency phase response.

5.2 High Overshoot from Z–N Tuning

The $\approx 61\%$ overshoot is consistent with the well-known aggressive nature of Z–N ultimate-period tuning, which minimises rise time rather than overshoot. For a position-control application this may be unacceptable. Reducing K_p and K_i while slightly increasing K_d, or using model-based pole-placement or loop-shaping, would allow an explicit phase- margin specification and lower overshoot.

5.3 Role of the Derivative Filter

Without the 200 Hz low-pass filter, the derivative term amplifies noise by a factor proportional to frequency. At the 1 kHz sampling rate, any noise above ~ 5 Hz would be differentiated, producing large spurious control signals. The 200 Hz filter attenuates noise by -20 dB/decade above the cutoff while introducing negligible phase lag at the ~ 5 Hz system bandwidth.

5.4 Anti-Windup Effectiveness

At step initiation the error is 1.0 V and the initial unsaturated control output is $K_p \times 1.0 + K_d \times$ (large derivative) $\gg 6$ V, so the actuator saturates immediately. Without anti-windup the integrator would accumulate to a large positive value during this interval, causing significantly worse overshoot on recovery. The conditional-update scheme freezes the integral during saturation, producing the cleaner settling visible in Figure 10.

5.5 Model Consistency

All numerical values in this report are mutually consistent:

- **Plant poles:** $(s + 35.09)(s + 40) \Rightarrow s^2 + 75.09s + 1403.6$; coefficients 75.09 and 1404 match the MATLAB output exactly.
- **Z–N gains:** $K_c = 8.710$, $T_c = 2\pi/37.465 = 0.16771$ s $\Rightarrow$ $K_p = 5.2264$, $K_i = 62.325$, $K_d = 0.10956$ — all match the `results3.pdf` printout.
- **Filtered PID:** Equation (4) coefficients (142.9, 6630, 78320) match the `C_pid_f` MATLAB output.
- **Discrete coefficients** in Tables 4 and 5 match the `results3.pdf` difference equation block for all three methods.
- **Step metrics** in Table 6 match the `stepinfo()` output in `results3.pdf` for Tustin, Forward Euler, and Backward Euler.

6 Conclusion

This experiment designed and implemented a digital PID position controller for a DC servomotor using the Ziegler–Nichols ultimate-period method and three discretization approaches. The key findings are:

- The Z–N method systematically identified $K_c = 8.710$ and $T_c = 0.1677$ s from the plant gain margin, yielding $K_p = 5.226$, $K_i = 62.325$, $K_d = 0.10956$.
- All three discretization methods (Tustin, Forward Euler, Backward Euler) produced stable, nearly identical responses at $T = 1$ ms because the sampling rate far exceeds the system bandwidth and the Z–N sampling criterion is satisfied ($10T/T_d = 0.477$).
- The 200 Hz first-order derivative low-pass filter is essential: it removes high-frequency noise from the derivative term without degrading low-frequency controller performance.
- Integrator anti-windup prevents excessive overshoot caused by integrator saturation during the initial large-error transient.
- The $\approx 61\%$ overshoot from Z–N tuning is inherent to the method and would require further gain adjustment or an alternative design strategy to reduce.
- Tustin discretization is the preferred method: it is the only transformation that guarantees stability for any sampling period and most faithfully preserves the continuous-time frequency response.

A C++ Controller Implementation

The program below implements both P-only and PID control with Tustin, Forward Euler, and Backward Euler discretizations. PID gains match the Z–N values derived in Section 2.3. Actuator saturation at ± 6 V and conditional anti-windup are included. The full source contains Tustin and Forward Euler variants following the same structure as the Backward Euler implementation shown here.

```
#define _USE_MATH_DEFINES
#include <iostream>
#include <stdio.h>
#include <conio.h>
#include <math.h>
#include "daq.h"

using namespace std;

// CONTROL_MODE: 0 = P only  |  1 = PID
// DISC_MODE:    0 = Tustin  |  1 = Forward Euler  |  2 = Backward Euler
int CONTROL_MODE = 1;
int DISC_MODE    = 2;   // Backward Euler active for this run

double reference = 1.0;          // position reference [V]
double T    = 0.001;             // sampling period [s]
double UMAX = 6.0;
double UMIN = -6.0;
double fc   = 200.0;             // derivative LPF cutoff [Hz]

// Z-N PID gains (Section 2.3 / Table 1)
double K  = 5.0;                 // P-only gain
double Kp = 5.226264;
double Ki = 62.325124;
double Kd = 0.109562;

// Controller state variables
double error_prev       = 0.0;
double integral         = 0.0;
double raw_deriv_prev   = 0.0;
double filt_deriv_prev  = 0.0;

double clamp(double v, double lo, double hi) {
    return (v > hi) ? hi : (v < lo ? lo : v);
}

// ---- Derivative filter: Backward Euler ----
// H(s) = wc/(s+wc)  -->  y[k] = a1*y[k-1] + b0*x[k]
double deriv_filter_backward(double raw) {
    double wc = 2.0 * M_PI * fc;
    double a1 = 1.0 / (1.0 + wc * T);          //  0.4432
    double b0 = (wc * T) / (1.0 + wc * T);     //  0.5568
    double y  = a1 * filt_deriv_prev + b0 * raw;
    return y;
}

// ---- PID: Backward Euler (DISC_MODE = 2) ----
double compute_pid_backward(double error) {
    // Proportional
    double P_out = Kp * error;

    // Integral (Backward Euler) with anti-windup candidate
    double integ_cand = integral + T * error;

```

```
    // Derivative: numerical difference then filtered
    double raw_d  = (error - error_prev) / T;
    double filt_d = deriv_filter_backward(raw_d);
    double D_out  = Kd * filt_d;

    // Total (unsaturated)
    double u_raw = P_out + Ki * integ_cand + D_out;
    double u_sat = clamp(u_raw, UMIN, UMAX);

    // Anti-windup: only commit integral if not saturated
    if (u_sat == u_raw) integral = integ_cand;

    // Update state
    error_prev      = error;
    raw_deriv_prev  = raw_d;
    filt_deriv_prev = filt_d;
    return u_sat;
}

// Dispatcher -- Tustin/Forward variants follow the same pattern
double compute_pid(double error) {
    if (DISC_MODE == 0) return compute_pid_tustin(error);
    if (DISC_MODE == 1) return compute_pid_forward(error);
    return compute_pid_backward(error);
}

int main() {
    AI_Configuration(1000, 10, -10, 10);
    AO_Configuration(-10, 10);

    bool running = true;
    printf("PID Control started. Press 'q' to stop.\n");

    while (running) {
        if (_kbhit() && _getch() == 'q') { running = false; break; }

        double y     = Read_Voltage();          // read position
        double error = reference - y;           // compute error

        double u = (CONTROL_MODE == 0)
                   ? clamp(K * error, UMIN, UMAX)
                   : compute_pid(error);

        Write_Voltage(u);

        printf("Ref: %.2f | y: %.3f | e: %.3f | u: %.3f\n",
               reference, y, error, u);
    }

    Write_Voltage(0.0);                         // safe stop
    printf("[SAFE STOP] Motor voltage = 0 V.\n");
    return 0;
```

```
}
```

Listing 1: Full C++ real-time PID controller (Backward Euler shown; Tustin and Forward Euler follow the same structure).